

Software Test Plan (STP) - Restaurant Management System (RMS)

Project: Restaurant Management System (RMS)

Version: 1.0

Authors: <QA Team>

Date: 02-10-2026

Status: Draft

1. Introduction

Purpose: This document defines the test plan for the Restaurant Management System (RMS) v1.0, a web-based application for table booking, food ordering and menu management for a single restaurant outlet. It outlines objectives, scope, strategy, resources, schedule, and responsibilities for testing, and follows the structure of IEEE Std 829 (Software Test Documentation).

Scope: Testing covers the four SRS system features - Table Booking Management, Menu Management, Food Ordering and Order Tracking, and User Authentication and Role Management - together with the SRS performance requirements (PERF-1 to PERF-3), security requirements (SEC-1 to SEC-8) and software quality attributes (QA-1 to QA-4). Testing spans the Django/DRF backend, the Node.js WebSocket service, the customer and admin web interfaces, and the shared relational database. Third-party libraries, hosting infrastructure and payment/SMS/email integration are excluded.

References: RMS SRS v1.1 (revised 28-09-2026, prepared by Abhay Dubey H, PES University), including Appendix C (Requirements Traceability Matrix) and Table 1 (Use case summary); IEEE Std 829 (Software Test Documentation); IEEE Std 830-1998 (SRS); Django and Node.js documentation; OWASP Top 10 / OWASP ASVS (security validation guidance).

Definitions: RMS (Restaurant Management System), SRS (Software Requirements Specification), RTM (Requirements Traceability Matrix), REQ (functional requirement), PERF (performance requirement), SEC (security requirement), SO (security objective), QA (software quality attribute), UC (use case), UT (Unit Test), IT (Integration Test), ST (System Test), SV (Security Validation test), RBAC (Role-Based Access Control), CSRF (Cross-Site Request Forgery), XSS (Cross-Site Scripting), TLS (Transport Layer Security), WSS (WebSocket Secure), UAT (User Acceptance Testing).

2. Test Items

- Booking module (Django app: booking) - availability search, booking, cancellation, staff booking management
- Menu module (Django app: menu) - menu CRUD, categories, availability filtering
- Ordering module (Django app: orders) - cart, order placement, order status transitions
- Authentication and access-control module (Django app: accounts) - registration, login, RBAC, session expiry, logout
- Real-time order-status service (Node.js / Socket.io: realtime/server.js)
- Audit logging component (core/audit.py)
- REST API layer (Django REST Framework, JSON over HTTPS)
- Customer web interface (Home/Menu, Table Booking, Cart/Checkout, Order-Tracking, Login/Sign-up pages)
- Admin/Staff dashboard (menu, booking and order management)
- Relational database (Users, Tables, Bookings, MenuItems, Orders, OrderItems)

3. Features to be Tested

Features mapped to SRS requirement IDs:

Table Booking Management (Priority: High)

- REQ-1: Display free tables for date, time slot and party size within 2 s
- REQ-2: Prevent double-booking by locking a table for its reserved slot
- REQ-3: Customer cancellation up to 30 minutes before the booking; table released
- REQ-4: Staff view, search, manually create and cancel bookings

Menu Management (Priority: High)

- REQ-5: Admin create, update, delete menu items
- REQ-6: Only items marked available shown to customers
- REQ-7: Exactly one category per item; items displayed grouped by category

Food Ordering and Order Tracking (Priority: High)

- REQ-8: Add available items to cart and submit as a single order
- REQ-9: Order status Received / Preparing / Served, updated by staff
- REQ-10: Status change pushed over WebSocket within 1 s of being saved
- REQ-11: Reject orders containing an item that became unavailable; notify customer which item was removed

User Authentication and Role Management (Priority: Medium)

- REQ-12: Register/login with email or phone and password (min. 8 chars); reject duplicates; salted PBKDF2-SHA256/Argon2 hash
- REQ-13: Role restrictions (menu management = Admin; booking override = Admin/Staff); others denied with HTTP 403
- REQ-14: Session/token invalidated after 30 minutes of inactivity

Non-functional requirements

- PERF-1: $\geq 95\%$ of page loads and API responses within 2 s at up to 50 concurrent users
- PERF-2: $\geq 95\%$ of WebSocket status updates delivered within 1 s
- PERF-3: 50 concurrent users with HTTP 5xx error rate $\leq 1\%$

- SEC-1 to SEC-8: Password hashing, TLS 1.2+, RBAC (401/403), SQL-injection/XSS protection, CSRF/token checks, account lockout, audit log, secure session cookies (detailed in Section 5.1)
 - QA-1: Usability (first-time booking within 3 min and <=6 page interactions)
 - QA-2: Maintainability (separate Django apps; no app imports another app's views)
 - QA-3: Reliability (confirmed bookings/orders persist across server restart)
 - QA-4: Portability (latest two stable versions of Chrome, Firefox, Edge)
- Business rules and safety (SRS 6.2 / 6.5): no overlapping reservation of a table; no order for unavailable items; staff may not delete a confirmed order or booking

4. Features Not to be Tested

- External payment gateway integration (not in scope of SRS v1.1; the "Payment Mode" field in the Order Report is verified as a data field only)
- SMS / email notification providers (not required in this version)
- Multi-branch / multi-tenant operation (single outlet only)
- Internationalization / localization (English only)
- Internals of third-party frameworks and libraries (Django, DRF, Socket.io, database engine) - assumed tested by their maintainers; only their configuration and use in RMS is tested
- Django built-in admin site internals (only RMS-specific admin screens and permissions are tested)
- Hosting infrastructure, operating system and network hardware

5. Test Approach / Strategy

Levels:

- Unit tests (UT_01 to UT_23, Appendix A): module-level tests of models, views, serializers and permissions using Django TestCase / pytest-django
- Integration tests (IT_01 to IT_10, Appendix B): REST API + database, Django <-> Node.js WebSocket service, role permissions across modules
- System tests (ST_01 to ST_14 and SR_01, Appendix C): end-to-end flows through the web UI and API, including the load test for PERF-1 and PERF-3
- Security validation (SV_01 to SV_10): see Section 5.1 and Appendix D
- Acceptance tests (UAT): scenario-based validation of use cases UC1 to UC10 by the Product Owner / evaluator

Types:

- Functional testing (all REQ items, positive and negative paths, boundary values such as 30-minute cancellation cut-off and 8-character password)
- Regression testing (full UT/IT suite on every build; ST suite before each milestone)
- Performance testing (response time, WebSocket latency, 50-user load)
- Concurrency testing (simultaneous booking attempts for the same table and slot)
- Usability testing (QA-1 booking task timing, inline form-validation messages)
- Compatibility testing (QA-4: Chrome, Firefox, Edge; desktop and mobile viewports)
- Reliability testing (QA-3: persistence across server restart)

Entry Criteria: Stable build delivered and deployed to the test environment, SRS v1.1 baselined, test data loaded, Django and Node.js services reachable, smoke test passed.
Exit Criteria: 100% of planned test cases executed, 0 critical/high defects open, all SV tests passed, PERF-1/2/3 thresholds met, 100% requirement coverage in the RTM, all acceptance criteria satisfied.

5.1 Security Validation

Security validation verifies requirements SEC-1 to SEC-8 against security objectives SO-1 to SO-4 of the SRS (Section 6.3). All security tests run only in the isolated test environment with test accounts. Full test case specifications (pre-conditions, steps, data) are in Appendix D.

- SO-1 Confidentiality: SV_01, SV_02, SV_09, SV_10
- SO-2 Integrity: SV_03, SV_04, SV_05, SV_06
- SO-3 Accountability: SV_03, SV_08
- SO-4 Availability: SV_07

Test ID	SRS Ref.	Validation / Technique	Expected Result
SV_01	SEC-1 REQ-12	Register two users with the same password; inspect stored hashes in the DB; search application logs and DB dumps for plain-text passwords.	Hashes use PBKDF2-SHA256 or Argon2 with different salts; no plain-text password in DB or logs.
SV_02	SEC-2	Scan the server with testssl.sh / nmap ssl-enum-ciphers; request pages over HTTP; open WebSocket over ws:// and wss://.	Only TLS 1.2+ accepted; HTTP redirected to HTTPS; WebSocket served over WSS only.
SV_03	SEC-3 REQ-13	Call every page/API endpoint without login, as Customer, as Staff and as Admin; attempt forced browsing and access to other users' bookings/orders by ID.	Unauthenticated = HTTP 401; wrong role = HTTP 403; menu management Admin-only; booking override Admin/Staff only.
SV_04	SEC-4	SQL-injection payloads (manual + sqlmap) on login, booking, search and menu fields.	Payloads neither executed nor cause errors; data unchanged; queries parameterised via ORM.
SV_05	SEC-4	XSS payloads (script tags, event handlers) in name, phone, menu description and order fields; view them in customer and admin pages.	Output is escaped; no script executes; stored data not rendered unescaped.
SV_06	SEC-5	Send POST/PUT/PATCH/DELETE requests without CSRF token or auth token (Postman / ZAP replay).	Every such request rejected with HTTP 403.
SV_07	SEC-6	Enter 5 consecutive wrong passwords; then try the correct password; retry after 15 minutes.	Account locked for 15 minutes after 5 failures; correct password rejected during lock; login works afterwards.
SV_08	SEC-7	Perform a menu change, a staff booking override and an order-status change; inspect audit log; check retention setting.	Each action logged with user ID, action, timestamp; entries retained for at least 90 days.
SV_09	SEC-8	Inspect session cookies in browser dev tools / ZAP.	Session cookies carry HttpOnly and Secure flags.
SV_10	SEC-8 REQ-14	Log in, stay idle for 30 minutes (test clock / reduced timeout), then perform an action.	Session invalidated after 30 minutes of inactivity; user must log in again.

Additional security activities:

- Fuzzing of input fields (booking form, registration, cart quantities, search) with OWASP ZAP / Burp Community to detect crashes and unhandled errors
- Penetration testing of authentication flows (login, registration, session handling, logout, token reuse after logout/expiry)
- Exploratory check that a customer receives only the status events of their own order over WebSocket
- Dependency vulnerability scan (pip audit / npm audit)

6. Test Environment

Hardware: Linux test server/VM (Python 3 + Node.js), client machines (desktop and mobile device) with a stable internet connection.

Software: RMS v1.0 build (Django + DRF backend, Node.js/Socket.io service), PostgreSQL (SQLite for demo-deployment checks), browsers Chrome, Firefox and Edge (latest two stable versions each).

Tools: pytest-django / Django TestCase (unit), Postman / Newman (API), Selenium (UI automation), JMeter or Locust (performance, 50 concurrent users), Socket.io test client (WebSocket latency), OWASP ZAP, sqlmap and testssl.sh (security), freezegun / test clock (time-based rules), Jira or GitHub Issues (defect tracking).

Test Data: Dummy Customer, Staff and Admin accounts; tables of different capacities; menu items in all four categories (Starters, Main Course, Desserts, Beverages) with both available and unavailable items; sample bookings and orders in each status; Appendix B field-layout boundary values.

7. Test Schedule

Milestones:

- Test case design: 05-Oct-2026
- Environment setup: 07-Oct-2026
- Test execution start: 08-Oct-2026
- Test execution end: 20-Oct-2026
- UAT: 22-Oct-2026 to 25-Oct-2026

8. Test Deliverables

- Test Plan (this document)
- Test Cases (manual & automated): UT_01-UT_23, IT_01-IT_10, ST_01-ST_14, SR_01, SV_01-SV_10 (Appendices A-D)
- Test Scripts
- Test Data
- Updated Requirements Traceability Matrix (SRS Appendix C)
- Test Execution Logs
- Defect Reports
- Security Validation Report
- Test Summary Report

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	<Name>	Prepare plan, coordinate execution
Test Engineer	<Name>	Design & execute test cases, log defects
Security Tester	<Name>	Execute SV tests, fuzzing and authentication penetration tests
Developer	<Name>	Support defect fixes and triage
Product Owner	<Name>	Approve test results, sign-off readiness

10. Risks and Mitigation

Risk	Mitigation
Delay in stable build delivery	Request early smoke builds from dev team
Test environment downtime	Maintain backup environment on cloud VM
Race conditions (double-booking) hard to reproduce	Use parallel-request scripts for REQ-2; add DB-level constraint checks
WebSocket timing results skewed by network latency	Measure on same host/LAN; repeat runs and report the 95th percentile (PERF-2)
Time-based rules (30-min idle, 15-min lockout, 30-min cancel cut-off, 90-day retention) are slow to test	Use configurable timeouts and a controllable test clock
Results on SQLite differ from PostgreSQL	Run performance and concurrency tests on PostgreSQL
Security tests corrupt test data	Run on isolated environment with resettable DB snapshot
Short academic project timeline	Prioritise High-priority features and SV tests first; automate regression

11. Assumptions & Dependencies

- SRS v1.1 is the approved baseline; changes are communicated before test case freeze
- Test data (accounts, tables, menu items) will be provided before execution
- Test environment provides HTTPS/WSS (self-signed certificate acceptable) and configurable session/lockout timeouts
- A booking occupies one time slot with a fixed turnaround (e.g., 90 minutes)
- The Node.js WebSocket library (Socket.io or equivalent) is available and stable
- No external payment gateway or SMS/email provider is integrated in this version

12. Suspension & Resumption Criteria

Suspend testing if:

- Environment unavailable for >4 hours
- Build is too unstable (blocks >30% test cases)
- A critical defect in login or database access blocks all dependent test cases

Resume testing if:

- Blocking defects are resolved
- Environment stabilized
- A new build passes the smoke test

13. Test Case Management & Traceability

RTM ensures mapping of SRS requirements to test cases. This plan reuses the test case IDs already assigned in SRS Appendix C (Requirements Traceability Matrix): UT = Unit, IT = Integration, ST = System, SV = Security Validation.

Requirement-to-test mapping:

Req. ID	Description	Unit	Integration	System	Security
REQ-1	Table availability within 2 s	UT_01, UT_02	IT_01	ST_01, ST_02	-
REQ-2	Prevent double-booking	UT_03, UT_04	IT_01	ST_01, ST_02	-
REQ-3	Cancel booking (30-min rule)	UT_05, UT_21, UT_23	-	ST_09	-
REQ-4	Staff manage bookings	-	IT_02, IT_03	ST_09	-
REQ-5	Admin menu CRUD	UT_06 to UT_09	IT_04	ST_03	-
REQ-6	Show only available items	UT_10	IT_04, IT_05	ST_04	-
REQ-7	Menu categories	UT_11	-	ST_03	-
REQ-8	Cart and order placement	UT_12, UT_13, UT_16	IT_08	ST_05	-
REQ-9	Order status transitions	UT_13, UT_14	-	ST_05	-
REQ-10	WebSocket push within 1 s	-	IT_06, IT_07	ST_05	-
REQ-11	Reject unavailable items	UT_15	-	ST_06	-
REQ-12	Registration / login	UT_17, UT_18, UT_19, UT_22	IT_10	ST_07	-
REQ-13	Role-based access control	-	IT_09	ST_08	SV_03
REQ-14	Session expiry (30 min)	UT_20	-	-	SV_10
PERF-1	95% responses within 2 s	-	-	ST_10	-

Req. ID	Description	Unit	Integration	System	Security
PERF-2	WebSocket within 1 s	-	IT_07	ST_05	-
PERF-3	50 users, <=1% errors	-	-	ST_10	-
SEC-1	Salted password hashing	UT_19	-	-	SV_01
SEC-2	TLS 1.2+ (HTTPS/WSS)	-	-	-	SV_02
SEC-3	RBAC with 401/403	-	IT_09	ST_08	SV_03
SEC-4	SQL injection / XSS	-	-	-	SV_04, SV_05
SEC-5	CSRF / token check	-	-	-	SV_06
SEC-6	Account lockout	-	-	-	SV_07
SEC-7	Audit log	-	-	-	SV_08
SEC-8	Secure session cookies, expiry	UT_20	-	-	SV_09, SV_10
QA-1	Usability (3 min / 6 steps)	-	-	ST_11*	-
QA-2	Maintainability (separate apps)	SR_01*	-	-	-
QA-3	Reliability (persistence)	-	-	ST_12*	-
QA-4	Portability (browsers)	-	-	ST_13*	-
BR	Staff cannot delete confirmed booking/order	-	-	ST_14*	-

Use case coverage (SRS Table 1):

Use Case	Requirements	System / Security Tests
UC1 Register / Login	REQ-12, 13, 14	ST_07, ST_08, SV_10
UC2 Check Table Availability	REQ-1	ST_01, ST_02
UC3 Book Table	REQ-1, 2	ST_01, ST_02
UC4 Cancel Booking	REQ-3	ST_09
UC5 Browse Menu	REQ-6, 7	ST_03, ST_04
UC6 Place Order	REQ-8, 11	ST_05, ST_06
UC7 Track Order Status	REQ-9, 10	ST_05
UC8 Manage Menu	REQ-5, 13	ST_03, ST_08, SV_03
UC9 Update Order Status	REQ-9, 13	ST_05, ST_08, SV_03
UC10 Manage Bookings	REQ-4, 13	ST_09, ST_08, SV_03

* Additional test cases written for SRS items that have no test ID in SRS Appendix C (to be added to the SRS RTM): UT_22 (REQ-12, password shorter than 8 characters), UT_23 (REQ-3, cancellation exactly 30 minutes before), ST_11 (QA-1), SR_01 (QA-2, static review), ST_12 (QA-3), ST_13 (QA-4), ST_14 (business rule: staff cannot delete a confirmed booking/order). Full specifications are in Appendices A and C.

Traceability rules: every test case records the requirement ID(s) it verifies; every

requirement must map to at least one test case; the RTM is updated whenever a requirement or test case changes.

14. Test Metrics & Reporting

Metrics collected:

- % test cases executed (51 baseline cases from SRS Appendix C = 21 UT + 10 IT + 10 ST + 10 SV, plus 7 additional cases = 58 planned)
- % passed/failed
- Defect density
- Defect aging
- Requirement coverage (REQ, PERF, SEC, QA mapped to executed test cases)
- Security: % SEC requirements validated, open vulnerabilities by severity
- Performance: 95th percentile response time (PERF-1), WebSocket latency (PERF-2), 5xx error rate (PERF-3)

Reports:

- Daily execution status
- Security Validation Report
- Final Test Summary Report

15. Approvals

Role	Name	Signature / Date
QA Lead		
Dev Lead		
Product Owner		

Appendix A: Unit Test Cases (UT_01 to UT_23)

Verify individual functions, models, serializers and permission checks in isolation (Django TestCase / pytest-django). UT_22 and UT_23 are additional cases not in SRS Appendix C.

Test Case ID	Name of Module	Test case description	Pre-conditions	Test Steps	Test data	Expected Results	Actual Result	Test Result
UT_01	Table Booking Management	Verify the system displays available tables for a valid date, time and party size (REQ-1)	Booking service is running; at least one table exists in the database	1: Call the availability-check function with a date, time slot and party size 2: Inspect the list of tables returned	Date: 2026-10-21, Time: 19:00, Party size: 4	Function returns all tables with capacity >= 4 that have no existing booking overlapping 19:00 on 2026-10-21		
UT_02	Table Booking Management	Verify the system returns no tables when none are free for the requested slot (REQ-1, negative)	All tables are already booked for the requested slot	1: Call the availability-check function for a fully-booked date/time/party size 2: Inspect the returned list	Date: 2026-10-21, Time: 19:00, Party size: 2 (slot fully booked)	Function returns an empty list and the system suggests the closest available slot		
UT_03	Table Booking Management	Verify creating a booking locks the selected table for its reserved slot (REQ-2)	Table 5 is free for the target slot	1: Submit a booking request for Table 5 at a given date/time 2: Query the table's status immediately after	Table: 5, Date: 2026-10-21, Time: 19:00, Party size: 2	Booking record is created and Table 5's status is 'Reserved' for that slot		
UT_04	Table Booking Management	Verify a booking cannot be created for a table already locked in an overlapping slot (REQ-2, negative)	Table 5 already has a confirmed booking for 19:00-20:30	1: Submit a second booking request for Table 5 at 19:30 on the same date 2: Observe the system response	Table: 5, Date: 2026-10-21, Time: 19:30 (overlaps existing 19:00-20:30 booking)	Booking request is rejected with a message indicating the table is unavailable for that slot		
UT_05	Table Booking Management	Verify a booking can be cancelled more than 30 minutes before the reserved time (REQ-3)	A confirmed booking exists for a slot more than 30 minutes away	1: Submit a cancellation request for the booking 2: Query the booking status and table status	Booking reserved for 19:00; cancellation submitted at 18:00	Booking status changes to 'Cancelled' and the table is released back into the available pool		
UT_06	Menu Management	Verify an admin can create a new menu item with valid name, price and category (REQ-5)	Admin is authenticated	1: Submit a create-menu-item request with valid name, price, category and availability 2: Query the menu items table for the new entry	Name: 'Paneer Tikka', Price: 220, Category: 'Starters', Available: true	Menu item is created successfully and appears in the menu items table with the given values		
UT_07	Menu Management	Verify menu item creation is rejected when the price is negative or a required field is missing (REQ-5, negative)	Admin is authenticated	1: Submit a create-menu-item request with a negative price and a missing name 2: Observe the system response	Name: "", Price: -50, Category: 'Starters'	Request is rejected with a validation error identifying the invalid/missing fields; no record is created		
UT_08	Menu Management	Verify an admin can update an existing menu item's details (REQ-5)	Menu item 'Paneer Tikka' already exists	1: Submit an update request changing the item's price and availability 2: Query the menu items table for the updated entry	Item: 'Paneer Tikka', New price: 240, Available: false	Menu item record is updated with the new price and availability status		

Test Case ID	Name of Module	Test case description	Pre-conditions	Test Steps	Test data	Expected Results	Actual Result	Test Result
UT_09	Menu Management	Verify an admin can delete a menu item (REQ-5)	Menu item 'Paneer Tikka' exists and is not referenced by an open order	1: Submit a delete request for the menu item 2: Query the menu items table	Item: 'Paneer Tikka'	Menu item is removed from the menu items table and no longer returned by menu queries		
UT_10	Menu Management	Verify only menu items marked 'available' are returned by the menu listing function (REQ-6)	Menu contains a mix of available and unavailable items	1: Call the menu-listing function 2: Inspect the returned list of items	5 items marked available, 2 items marked unavailable	Only the 5 available items are returned; the 2 unavailable items are excluded		
UT_11	Menu Management	Verify menu items are grouped correctly by category (REQ-7)	Menu contains items across Starters, Main Course and Desserts categories	1: Call the menu-listing function with grouping enabled 2: Inspect the grouped output	3 Starters, 4 Main Course, 2 Desserts items	Items are grouped under their correct category headings with correct counts in each group		
UT_12	Food Ordering and Order Tracking	Verify a customer can add one or more available menu items to the cart (REQ-8)	At least 2 menu items are marked available	1: Add 'Paneer Tikka' to the cart 2: Add 'Spring Rolls' to the cart 3: Inspect the cart contents	Items: Paneer Tikka (x1), Spring Rolls (x2)	Cart contains both items with the correct quantities		
UT_13	Food Ordering and Order Tracking	Verify submitting a cart creates a single order with status 'Received' (REQ-8, REQ-9)	Cart contains at least one available item	1: Submit the cart as an order 2: Query the orders table for the new record	Cart: Paneer Tikka (x1), Spring Rolls (x2)	A single order record is created containing both items with status 'Received'		
UT_14	Food Ordering and Order Tracking	Verify staff can update an order's status from Received to Preparing to Served (REQ-9)	An order exists with status 'Received'	1: Staff updates the order status to 'Preparing' 2: Staff updates the order status to 'Served' 3: Query the order's status after each update	Order ID: OR-2001	Order status changes to 'Preparing' and then to 'Served' as each update is applied		
UT_15	Food Ordering and Order Tracking	Verify an order is rejected if it contains a menu item marked unavailable, with correct notification (REQ-11, negative)	One item in the cart has been marked unavailable after being added	1: Add an item to the cart while it is available 2: Admin marks the item unavailable 3: Submit the cart as an order	Cart contains 'Spring Rolls', which is then marked unavailable	Order submission fails; customer is notified that 'Spring Rolls' is no longer available and was removed		
UT_16	Food Ordering and Order Tracking	Verify the cart correctly calculates the total price for multiple items and quantities (REQ-8)	Menu item prices are known	1: Add Paneer Tikka (price 220) x1 and Spring Rolls (price 180) x2 to the cart 2: Inspect the cart's calculated total	Paneer Tikka x1 (220), Spring Rolls x2 (180 each)	Cart total is correctly calculated as $220 + (180 \times 2) = 580$		
UT_17	User Authentication and Role Management	Verify a new customer can register with a valid email/phone and password (REQ-12)	The email/phone used is not already registered	1: Submit the registration form with a valid email and password 2: Query the Users table for the new account	Email: customer1@example.com, Password: Cust@1234	A new user account is created with role 'Customer' and the password is stored hashed, not in plain text		

Test Case ID	Name of Module	Test case description	Pre-conditions	Test Steps	Test data	Expected Results	Actual Result	Test Result
UT_18	User Authentication and Role Management	Verify registration is rejected for an email that is already registered (REQ-12, negative)	An account with the same email already exists	1: Submit the registration form with an email that is already registered 2: Observe the system response	Email: customer1@example.com (already registered), Password: Cust@1234 (valid)	Registration is rejected with a validation error indicating the email is already in use		
UT_19	User Authentication and Role Management	Verify passwords are stored using a secure salted hash and never in plain text (REQ-12, SEC-1)	A user account has just been created	1: Register a new user 2: Inspect the stored password value directly in the database	Email: customer2@example.com, Password: Cust@5678	The stored value is a hashed string (e.g., PBKDF2/Argon2 format); the plain-text password is not present		
UT_20	User Authentication and Role Management	Verify a logged-in user's session/token is invalidated after 30 minutes of inactivity (REQ-14, SEC-8)	User is logged in and holds a valid session/token	1: Log in and note the issued session/token 2: Remain inactive for 30 minutes (test clock or reduced timeout in the test environment) 3: Attempt to access a protected page using the same token	Inactivity period: 30 minutes	The session/token is rejected as expired/invalid and the user is prompted to log in again		
UT_21	Table Booking Management	Verify a booking cannot be cancelled less than 30 minutes before the reserved time (REQ-3, negative)	A confirmed booking exists for 19:00	1: Submit a cancellation request at 18:45 2: Query the booking and table status	Booking reserved for 19:00; cancellation submitted at 18:45	Cancellation is rejected with a message that bookings can be cancelled only up to 30 minutes before the reserved time; booking status remains 'Confirmed' and the table stays reserved		
UT_22	User Authentication and Role Management	Verify registration is rejected when the password is shorter than 8 characters (REQ-12, negative)	The email/phone used is not already registered	1: Submit the registration form with a valid email and a 7-character password 2: Observe the system response 3: Query the Users table	Email: customer4@example.com, Password: Ab@1234 (7 characters)	Registration is rejected with a validation error stating the minimum length of 8 characters; no account is created		
UT_23	Table Booking Management	Verify a booking can be cancelled exactly 30 minutes before the reserved time (REQ-3, boundary)	A confirmed booking exists for 19:00	1: Submit a cancellation request at exactly 18:30 2: Query the booking status and table status	Booking reserved for 19:00; cancellation submitted at 18:30	Cancellation is accepted; booking status changes to 'Cancelled' and the table is released back into the available pool		

Appendix B: Integration Test Cases (IT_01 to IT_10)

Verify interaction between the Django REST API, the database and the Node.js/Socket.io real-time service.

Test Case ID	Name of Module	Test case description	Pre-conditions	Test Steps	Test data	Expected Results	Actual Result	Test Result
IT_01	Table Booking Management	Verify the booking API persists a new booking to the database and updates table status (REQ-1, REQ-2)	Booking API and database are both running and reachable	1: Send a POST request to the booking endpoint with valid date/time/party size/table 2: Query the database directly for the new booking record 3: Query the table's status in the database	POST /api/bookings {table:5, date:'2026-10-21', time:'19:00', guests:2}	API returns 201 Created; a matching row exists in the Bookings table; Table 5's status is 'Reserved'		
IT_02	Table Booking Management	Verify the staff dashboard retrieves and displays current bookings from the booking service (REQ-4)	At least 3 bookings exist for the current day; staff user is logged in	1: Log in as a Staff user 2: Open the Bookings section of the admin dashboard 3: Compare the list shown against the database records	Staff credentials; 3 seeded bookings for today	All 3 bookings are listed on the dashboard with correct table, time and customer details		
IT_03	Table Booking Management	Verify staff can manually cancel a booking from the dashboard and the change reflects in the database (REQ-4)	A confirmed booking exists; staff user is logged in	1: Log in as Staff 2: Open the booking in the dashboard and select 'Cancel Booking' 3: Query the database for the booking's status	Existing booking ID: BK-1001	Dashboard shows the booking as 'Cancelled'; database record for BK-1001 has status 'Cancelled'		
IT_04	Menu Management	Verify a menu update made via the admin panel is immediately reflected in the customer-facing menu API (REQ-5, REQ-6)	Admin is logged in; customer-facing menu API is reachable	1: Update an existing item's price via the admin panel 2: Call the customer-facing menu API for the same item	Item: 'Paneer Tikka', updated price: 250	Customer-facing API returns the item with the updated price of 250, with no manual refresh needed		
IT_05	Menu Management	Verify marking an item unavailable removes it from the customer menu listing returned by the API (REQ-6)	Item is currently available and visible to customers	1: Mark the item as unavailable via the admin panel 2: Call the customer-facing menu listing API	Item: 'Paneer Tikka' marked unavailable	The item no longer appears in the customer-facing menu API response		
IT_06	Food Ordering and Order Tracking	Verify an order-status change in the backend triggers a WebSocket push event to the Node.js real-time service (REQ-10)	Node.js WebSocket service is running and connected to the Django backend	1: Update an order's status via the staff dashboard 2: Inspect the WebSocket event emitted by the Node.js service	Order ID: OR-2001, new status: 'Preparing'	A WebSocket event is emitted containing the order ID and the new status 'Preparing'		
IT_07	Food Ordering and Order Tracking	Verify the customer's order-tracking screen updates in real time upon receiving a WebSocket status push (REQ-10)	Customer's order-tracking page is open with an active WebSocket connection	1: Trigger an order-status update on the backend 2: Observe the customer's order-tracking screen without refreshing	Order ID: OR-2001, status change: 'Received' -> 'Preparing'	The order-tracking screen updates to show 'Preparing' within 1 second, with no manual refresh		
IT_08	Food Ordering and Order Tracking	Verify order submission correctly persists the order and order items in the database with correct	Cart contains valid available items	1: Submit the cart as an order 2: Query the Orders and OrderItems tables	Cart: Paneer Tikka x1, Spring Rolls x2	Orders table has one new row; OrderItems table has two rows linked to it; total amount matches cart total		

Test Case ID	Name of Module	Test case description	Pre-conditions	Test Steps	Test data	Expected Results	Actual Result	Test Result
		totals (REQ-8)						
IT_09	User Authentication and Role Management	Verify an Admin-role user can access menu-management endpoints while a Customer-role user is denied (REQ-13)	One Admin account and one Customer account exist	1: Log in as Admin and call the menu-management endpoint 2: Log in as Customer and call the same endpoint	Admin credentials; Customer credentials	Admin request succeeds (200 OK); Customer request is denied (403 Forbidden)		
IT_10	User Authentication and Role Management	Verify login issues a valid session/token that is accepted by protected API endpoints (REQ-12, REQ-13)	A registered user account exists	1: Log in with valid credentials and capture the issued token 2: Call a protected endpoint using that token	Email: customer2@example.com, Password: Cust@5678	Login succeeds and returns a valid token; the protected endpoint accepts the token and returns the expected data		

Appendix C: System Test Cases (ST_01 to ST_14, SR_01)

Verify complete end-to-end scenarios through the web UI and API, including the load test for PERF-1 and PERF-3. ST_11 to ST_14 and SR_01 are additional cases not in SRS Appendix C.

Test Case ID	Name of Module	Test case description	Pre-conditions	Test Steps	Test data	Expected Results	Actual Result	Test Result
ST_01	Table Booking Management	End-to-end: customer books a table via the UI; available tables are shown within 2 seconds and a confirmation with a booking reference is displayed (REQ-1, REQ-2)	Customer is on the Table Booking page; at least one table is free for the chosen slot	1: Select a date, time and party size 2: Choose a table from the available options and confirm 3: Observe the confirmation screen	Date: 2026-10-22, Time: 20:00, Party size: 3	Available tables are displayed within 2 seconds of the request; booking is confirmed and a unique booking reference number is displayed to the customer		
ST_02	Table Booking Management	End-to-end: customer attempts to double-book an already reserved slot; system rejects and suggests the nearest available slot (REQ-1, REQ-2)	The chosen table/slot combination is already fully booked	1: Select a date, time and party size that is already fully booked 2: Attempt to confirm the booking 3: Observe the system's response	Date: 2026-10-22, Time: 20:00 (already fully booked), Party size: 3	Booking is rejected with a message, and the system suggests the closest available time slot; the availability response is displayed within 2 seconds		
ST_03	Menu Management	End-to-end: admin adds a new Starters item; customer browsing the menu sees it under Starters (REQ-5, REQ-7)	Admin is logged in; customer has the menu page open	1: Admin adds a new item 'Spring Rolls' under category Starters 2: Customer refreshes/browses the menu page 3: Customer looks under the Starters category	Item: 'Spring Rolls', Category: 'Starters', Price: 180	'Spring Rolls' appears under the Starters category on the customer-facing menu		

Test Case ID	Name of Module	Test case description	Pre-conditions	Test Steps	Test data	Expected Results	Actual Result	Test Result
ST_04	Menu Management	End-to-end: admin marks an item unavailable; customer no longer sees it while browsing the live menu (REQ-6)	Item is currently visible to the customer	1: Admin marks the item as unavailable 2: Customer browses/refreshes the menu page	Item: 'Spring Rolls' marked unavailable	'Spring Rolls' no longer appears on the customer-facing menu		
ST_05	Food Ordering and Order Tracking	End-to-end: customer places an order, staff updates status through all stages, customer sees live updates within 1 second of each change (REQ-8, REQ-9, REQ-10, PERF-2)	Customer has items in the cart; staff dashboard is open	1: Customer submits the order 2: Staff updates status to 'Preparing', then 'Served' 3: Customer observes their order-tracking screen throughout	Cart: Paneer Tikka x1, Spring Rolls x2	Customer's screen updates live through Received -> Preparing -> Served without any page refresh, each update appearing within 1 second of the status change being saved		
ST_06	Food Ordering and Order Tracking	End-to-end: customer attempts to order an item that becomes unavailable after adding it to the cart (REQ-11)	Item is available when added to the cart	1: Customer adds 'Spring Rolls' to the cart 2: Admin marks 'Spring Rolls' unavailable before checkout 3: Customer submits the order	Cart: Spring Rolls x2, marked unavailable before submission	Order submission is blocked and the customer is notified that 'Spring Rolls' was removed as unavailable		
ST_07	User Authentication and Role Management	End-to-end: customer registers, logs in, and books a table using their authenticated session (REQ-12)	Email used for registration is not already registered	1: Register a new customer account 2: Log in with the new account 3: Use the authenticated session to book a table	Email: customer3@example.com, Password: Cust@9012	Registration, login and booking all succeed, with the booking correctly associated with the new account		
ST_08	User Authentication and Role Management	End-to-end: a Staff-role user attempts to access an Admin-only menu-management page and is denied access (REQ-13)	A Staff account exists and is logged in	1: Log in as Staff 2: Navigate directly to the Admin-only menu-management page	Staff credentials	Access is denied with HTTP 403 Forbidden and an appropriate message; the menu-management page is not displayed		
ST_09	Table Booking Management	End-to-end: staff creates a booking for a walk-in customer from the dashboard and cancels it more than 30 minutes ahead (REQ-3, REQ-4)	Staff user is logged in; Table 3 is free for the slot	1: Log in as Staff 2: Create a booking for Table 3 via the dashboard 3: Verify it appears in the booking list 4: Cancel it from the dashboard	Table 3, 2026-10-23, 19:00, guests: 2, customer: Walk-in Guest	Booking is created and listed as 'Confirmed'; after cancellation it shows 'Cancelled' and Table 3 is available again for that slot		
ST_10	Performance (NFR)	Load test: 50 concurrent users browse the menu, book tables and place orders (PERF-1, PERF-3)	System deployed on the test server with seeded data; load tool (Locust/JMeter) configured	1: Ramp up to 50 virtual users over 1 minute 2: Each user browses the menu, checks availability and places an order 3: Run for 10 minutes 4: Collect response-time and error statistics	50 concurrent virtual users, 10-minute run	At least 95% of page loads/API responses complete within 2 s; HTTP 5xx error rate is 1% or less		
ST_11	Usability	First-time customer	Tester has never	1: Give the tester the task: book a	3 first-time testers;	Each tester completes the		

Test Case ID	Name of Module	Test case description	Pre-conditions	Test Steps	Test data	Expected Results	Actual Result	Test Result
	(NFR)	completes a table booking within 3 minutes and no more than 6 page interactions without training (QA-1)	used RMS; test account exists; at least one table is free	table for 2 guests tomorrow at 19:00 2: Time the task with a stopwatch and count page interactions 3: Record the result for each tester	Party size: 2	booking in 3 minutes or less using no more than 6 page interactions		
ST_12	Reliability (NFR)	Confirmed bookings and orders persist across a server restart (QA-3)	A confirmed booking and a placed order exist	1: Note the booking reference and order ID 2: Restart the Django and Node.js services 3: View the booking and order again via the UI and database	Booking ID: BK-1001, Order ID: OR-2001	Both records are present with unchanged details and status after restart; nothing is lost		
ST_13	Portability (NFR)	Core flows work in the latest two stable versions of Chrome, Firefox and Edge (QA-4)	Test environment accessible from all target browsers	1: In each browser version, log in 2: Browse the menu, book a table, place an order and open order tracking 3: Note any rendering or functional defects	Chrome, Firefox, Edge (latest two stable versions each)	All pages render and function correctly in every listed browser version		
ST_14	Business Rules	Staff cannot delete a confirmed booking or order (SRS 6.5)	Staff user is logged in; confirmed booking BK-1001 and order OR-2001 exist	1: Attempt to delete the booking from the dashboard and via the API 2: Attempt to delete the order from the dashboard and via the API 3: Cancel the booking (status change) from the dashboard	Staff credentials; BK-1001; OR-2001	Deletion is not offered or is denied (HTTP 403) and both records remain; cancelling the booking is still allowed and sets status 'Cancelled'		
SR_01	Maintainability (static review)	Verify no Django app imports another app's views (QA-2)	Source code available; apps booking, menu, orders, accounts exist	1: List import statements in each app (grep / import-linter) 2: Check that no app imports views from another app	Source code of the 4 Django apps	No cross-app view imports are found; code base is split into the 4 separate apps		

Appendix D: Security Validation Test Cases (SV_01 to SV_10)

Detailed specifications of the security tests summarised in Section 5.1.

Test Case ID	Name of Module	Test case description	Pre-conditions	Test Steps	Test data	Expected Results	Actual Result	Test Result
SV_01	Security - SEC-1	Verify passwords are stored only as salted PBKDF2-SHA256/Argon2 hashes and never appear	Test environment with database and log access	1: Register two customers with the same password 2: Inspect both stored password values in the database	customer_sv1a@example.com and customer_sv1b@example.com,	Both values are PBKDF2-SHA256 or Argon2 hashes with different salts; the plaintext password is absent from		

Test Case ID	Name of Module	Test case description	Pre-conditions	Test Steps	Test data	Expected Results	Actual Result	Test Result
		in plain text (SEC-1, REQ-12)		3: Search application logs and a DB dump for the plain-text password	Password: Same@1234	DB and logs		
SV_02	Security - SEC-2	Verify all traffic uses TLS 1.2+ and HTTP is redirected to HTTPS (SEC-2)	Test server deployed with TLS certificate	1: Scan with testssl.sh / nmap ssl-enum-ciphers 2: Request the site over http:// 3: Open the WebSocket over ws:// and wss://	Test server hostname	Only TLS 1.2 or higher is accepted; HTTP is redirected to HTTPS; WebSocket works only over WSS		
SV_03	Security - SEC-3 / REQ-13	Verify role-based access control on every page and endpoint returns 401 (unauthenticated) and 403 (wrong role) (SEC-3, REQ-13)	Customer, Staff and Admin accounts exist; bookings/orders of another user exist	1: Call every page/API endpoint without logging in 2: Repeat as Customer, Staff and Admin 3: Attempt forced browsing and access to another user's booking/order by ID	Role credentials; list of all endpoints	Unauthenticated requests get HTTP 401; unauthorised roles get HTTP 403; menu management is Admin-only; booking override is Admin/Staff only		
SV_04	Security - SEC-4	Verify SQL-injection payloads are not executed (SEC-4)	Test environment with resettable DB snapshot	1: Enter SQL-injection payloads in login, booking, search and menu fields (manual + sqlmap) 2: Observe responses and check the database	' OR '1'='1 ; '; DROP TABLE bookings;--	Payloads are not executed; no data is changed or leaked; queries go through the ORM/parameterised queries		
SV_05	Security - SEC-4	Verify XSS payloads are escaped on output and not executed (SEC-4)	Customer and Admin accounts exist	1: Enter script payloads in name, phone, menu description and order fields 2: View the stored data in customer and admin pages	<script>alert(1)</script> ; 	Output is escaped; no script executes; stored data is not rendered unescaped		
SV_06	Security - SEC-5	Verify state-changing requests without a valid CSRF/auth token are rejected (SEC-5)	Test accounts exist; request-replay tool (Postman/ZAP) ready	1: Send POST/PUT/PATCH/DELETE requests with no CSRF token 2: Send the same requests with no authentication token	Booking, order and menu endpoints	Every such request is rejected with HTTP 403		
SV_07	Security - SEC-6	Verify account lockout for 15 minutes after 5 consecutive failed logins (SEC-6)	A registered customer account exists	1: Enter a wrong password 5 times in a row 2: Enter the correct password 3: Retry after 15 minutes (test clock)	customer2@example.com; wrong password: Wrong@123	Account is locked after the 5th failure; correct password is rejected during the lock; login succeeds after 15 minutes		
SV_08	Security - SEC-7	Verify an audit-log entry is written for privileged actions and kept for at least 90 days (SEC-7)	Admin and Staff users logged in; audit logging enabled	1: Change a menu item as Admin 2: Override (create/cancel) a booking as Staff 3: Change an order status as Staff 4: Inspect the audit log and the retention setting	Item: Paneer Tikka; BK-1001; OR-2001	Each action has an entry with user ID, action and timestamp; retention is configured as at least 90 days		
SV_09	Security - SEC-8	Verify session cookies carry the HttpOnly and	A registered user account exists	1: Log in over HTTPS 2: Inspect the session cookie in	customer2@example.com	Session cookie has both HttpOnly and Secure flags set		

Test Case ID	Name of Module	Test case description	Pre-conditions	Test Steps	Test data	Expected Results	Actual Result	Test Result
		Secure flags (SEC-8)		browser dev tools / ZAP				
SV_10	Security - SEC-8 / REQ-14	Verify the session is invalidated after 30 minutes of inactivity (SEC-8, REQ-14)	User is logged in with a valid session	1: Remain idle for 30 minutes (test clock or reduced timeout) 2: Perform an action in the same session	customer2@example.com; idle time: 30 minutes	Session is invalidated and the user is required to log in again		